Public Key Infrastructure

Public Keys are very useful

Secure web connections
Software signing
Secure messaging, email
Cryptocurrency, blockchains
How do we know the public key of an entity? And how can we trust that this entity is indeed who they claim to be and that they own a specific public key? Mainly: the key must be signed by a trusted Certificate Authority (CA)
Public Key Infrastructure (PKI) defines how to issue, manage, and use such certificates

Public Key Certificates & Authorities

The big picture: when receiving a party's (the subject's) public key, it will be accompanied with a certificate. A valid cert means that the entity is who they claim to be and they own the corresponding public key
Certificate: signature by a CA over subject's public key and attributes
Attributes: identity (ID) and others (e.g. location)

- Validated by CA (liability?)
- Used by relying party for decisions

Root CA is most trusted in the hierarchy

Certificates are all about Trust

Rogue Certs

Rogue certificate: certificates that contain wrong or misleading information (so they *should* fail PKI validation)
Attacker goals:

- Impersonate: website, phishing email, signed malware...
- Equivocating (same name): circumvent name-based security mechanisms such as blacklists, access-control..

Types of misleading names:

- Combo names: bank.com vs accts-bank.com, bank.accts.com
- Domain-name hacking: accts.bank.com vs accts-bank.com, accts.bank.co
- Homographic: paypal.com vs paypaI.com
- Typo-squatting: banc.com, baank.com

PKI Failures

Although the signature over the cert verifies correctly, there is still a failure and the cert may be revoked: this is called a PKI failure
PKI failures include:

- Corrupt CA
- Validation failure
- Exposed CA private key
- Cryptanalysis cert forgery: find collisions in HtS paradigm, or exploit some vulnerability in the digital signature scheme used for signing

X.509 Certificates

The X.509 Standard Certificate Format

Idea: signature binds public key to distinguished name (DN) and to other attributes
Used widely despite complaints about its complexity: SSL/TLS, code-signing, IP-Sec, ...
Signed Fields:

1. Version
2. Cert serial number
3. Signature Algorithm Object Identifier (OID)
4. Issuer Distinguished Name (DN)
5. Validity period (make sure it's time-limited)
6. Subject (user) DN
7. Subject public key info

Then, signature

X.509 Certs and Subject Identifiers

V1: Distinguished Name (for subject and issuer) V2: Unique Identifiers (for subject and issuer) V3: Extensions (used in practice)

Distinguished Names

Most certs contain identifiers
Influenced by telecom providers: phone directory services are based on common names
Basic goals of identifiers:

- Meaningful (to humans)
- Unique identification of entity (owner)
- Decentralized, with accountability

The Identifiers Trilemma

Achieving the three goals: Meaningful, Unique, Decentralized, seems very challenging
Examples of achieving any <u>two</u> of the goals:

- Unique + Meaningful: URL, email
- Meaningful + Decentralized: common name
- Unique + Decentralized: hash of key

X.509 Validation of Certificate Paths

Have Root CA, with ICAs (intermediary CA) and LCAs (leaf CAs)
Only leaf CAs interact with end users.
Simply validate all certs in the chain all the way to the root CA
The root CA (self-signed) cert is in the root store in Alice's browser

Certificate Revocation

Reasons for revoking:

- Security issues: key compromised, CA compromised
- Administrative issues: change name, location

Techniques

Certificate Revocation List (CRL) A CRL is simply a list of revoked certs, distributed periodically by CAs
If CRLs contain all revoked certs (which did not expire) it could be huge!
CRLs are not immediate: who is responsible until CRL is distributed. Frequent CRLs → even more overhead

CRL Optimization Solutions

CRL distribution point: split certs to several CRLs
Authorities Revocation List (ARL): list only revoked CAs (very coarse grain, revoking CAs as a whole but not necessarily individuals)
Delta CRL: only new revocations since last "base" CRL– need to keep CRLs for a long period to check deltas
Another more efficient approach is the Online Certificate Status Protocol (OCSP)

Online Certificate Status Protocol (OCSP)

Improve efficiency and freshness compared to CRLs
Client asks CA about cert during handshake
CA signs response (real-time)
Adds additional delay to access time
Tradeoff: less cost, but higher latency

OCSP Challenges

Privacy (expose domain and client to CA), load on CA, response delay, reliability (what if CA fails)
Ambiguity: when an OCSP server (or CA) cannot resolve the request, it replies with "certificate status is unknown"
Reliability or failed requests: client failed to establish connection with OCSP server, or client's request is invalid (not signed, or not authorized) so no response will be received.

Ambiguous/Failed OCSP Responses

What should client do?
Wait forever? Unrealistic
Hard-fail: terminate connection since certificate is unknown/not received (safe!)
Ask user: application displaying message asking user how they want to proceed
Soft-fail: pretend that a response has been received and continue as if the cert is not revoked (common choice for browsers, but a MitM attacker may block the OCSP response to make a revoked cert go through)

Conclusion

PKI is an essential component of the internet
Yet, it is a complicated module with many issues related to security, privacy, and performance

- To many, this is a solved problem (but that is not the case)
- Several open questions related on how to detect rogue certs, handle CA failure, revocation ,etc, how to audit these parties, reduce trust..
- How to handle all these issues efficiently? We all want a highly responsive Web!